



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Settings UI

The Settings UI allows users to configure your plugin through Noctalia's settings panel. When users navigate to Plugins > Your Plugin > Configure, your Settings component is displayed.

Basic Structure

A Settings UI component is a QML file that must follow this structure:

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Widgets

ColumnLayout {
    id: root

    // Plugin API (injected by the settings dialog system)
    property var pluginApi: null

    // Local state for editing
    property string editValue: pluginApi?.pluginSettings?.myValue || ""

    spacing: Style.marginM

    // Your settings controls here

    // Required: Save function called by the dialog
    function saveSettings() {
        pluginApi.pluginSettings.myValue = root.editValue
        pluginApi.saveSettings()
    }
}
```

Required Elements

pluginApi

Injected by the settings dialog system. Provides access to plugin settings and the save function.

```
property var pluginApi: null
```

saveSettings() Function

Required. This function is called automatically when the user clicks “Save” in the settings dialog.

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
function saveSettings() {  
    // Update plugin settings  
    pluginApi.pluginSettings.myValue = root.editValue  
    pluginApi.pluginSettings.otherValue = root.otherEdit  
  
    // Persist to disk  
    pluginApi.saveSettings()  
  
    Logger.i("MyPlugin", "Settings saved")  
}
```

Manifest Entry

Register the settings UI in your plugin’s manifest.json:

```
{  
  "id": "my-plugin",  
  "name": "My Plugin",  
  "version": "1.0.0",  
  "entryPoints": {  
    "barWidget": "BarWidget.qml",  
    "settings": "Settings.qml"  
  },  
  "metadata": {  
    "defaultSettings": {  
      "message": "Hello World",  
      "enabled": true,  
      "count": 0  
    }  
  }  
}
```

Local State Pattern

Always use local state variables to track changes before saving. This allows users to cancel without applying changes:

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
ColumnLayout {  
  id: root  
  
  property var pluginApi: null  
  
  // Local state - initialized from saved settings  
  property string editMessage:  
    pluginApi?.pluginSettings?.message ||  
    pluginApi?.manifest?.metadata?.defaultSettings?.message ||  
    ""  
  
  property bool editEnabled:  
    pluginApi?.pluginSettings?.enabled ??  
    pluginApi?.manifest?.metadata?.defaultSettings?.enabled ??  
    true  
  
  property int editCount:  
    pluginApi?.pluginSettings?.count ||  
    pluginApi?.manifest?.metadata?.defaultSettings?.count ||  
    0  
  
  // Controls update local state, not settings directly  
  NTextInput {  
    text: root.editMessage  
    onTextChanged: root.editMessage = text  
  }  
  
  // Save applies local state to settings  
  function saveSettings() {  
    pluginApi.pluginSettings.message = root.editMessage  
    pluginApi.pluginSettings.enabled = root.editEnabled  
    pluginApi.pluginSettings.count = root.editCount  
    pluginApi.saveSettings()  
  }  
}
```

Available Widgets

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Text Input

```
NTextInput {  
  Layout.fillWidth: true  
  label: "Display Message"  
  description: "The message shown in the widget"  
  placeholderText: "Enter a message..."  
  text: root.editMessage  
  onTextChanged: root.editMessage = text  
}
```

Toggle Switch

```
NToggle {  
  Layout.fillWidth: true  
  label: "Enable Feature"  
  description: "Turn this feature on or off"  
  checked: root.editEnabled  
  onCheckedChanged: root.editEnabled = checked  
}
```

Checkbox

```
NCheckbox {  
  text: "Show notifications"  
  checked: root.editNotifications  
  onCheckedChanged: root.editNotifications = checked  
}
```

Slider

```
ColumnLayout {  
  Layout.fillWidth: true
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NLabel {  
  label: "Opacity"  
  description: "Widget transparency level"  
}  
  
NSlider {  
  Layout.fillWidth: true  
  from: 0  
  to: 100  
  value: root.editOpacity  
  onValueChanged: root.editOpacity = value  
}  
}
```

Spin Box

```
ColumnLayout {  
  Layout.fillWidth: true  
  spacing: Style.marginS  
  
  NLabel {  
    label: "Refresh Interval"  
    description: "Update frequency in seconds"  
  }  
  
  NSpinBox {  
    from: 1  
    to: 60  
    value: root.editInterval  
    onValueChanged: root.editInterval = value  
  }  
}
```

Combo Box

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Layout.minWidth: **true**

spacing: **Style.marginS**

```
NLabel {  
  label: "Display Mode"  
  description: "How to show the widget"  
}
```

```
NComboBox {  
  Layout.fillWidth: true  
  model: ["Compact", "Normal", "Expanded"]  
  currentIndex: root.editModelIndex  
  onCurrentIndexChanged: root.editModelIndex = currentIndex  
}  
}
```

Color Picker

```
ColumnLayout {  
  Layout.fillWidth: true
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NLabel {  
  label: "Accent Color"  
  description: "Custom color for highlights"  
}  
  
NColorPicker {  
  Layout.preferredWidth: Style.sliderWidth  
  Layout.preferredHeight: Style.baseWidgetSize  
  selectedColor: root.editColor  
  onColorSelected: function(color) {  
    root.editColor = color  
  }  
}  
}
```

Label (No Input)

```
NLabel {  
  label: "Section Title"  
  description: "Optional description text"  
}
```

Layout and Styling

Spacing

Use consistent spacing throughout your settings:


```
ColumnLayout {  
    id: root
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
spacing: Style.marginM // Standard spacing between controls
```

```
// Group related settings
```

```
ColumnLayout {  
    Layout.fillWidth: true  
    spacing: Style.marginS // Tighter spacing within groups
```

```
NLabel {  
    label: "Appearance"  
}
```

```
NToggle {  
    // ...  
}
```

```
NColorPicker {  
    // ...  
}  
}
```

```
// Divider between sections
```

```
NDivider {  
    Layout.fillWidth: true  
    Layout.topMargin: Style.marginM  
    Layout.bottomMargin: Style.marginM  
}
```

```
// Another section
```

```
ColumnLayout {  
    Layout.fillWidth: true  
    spacing: Style.marginS
```

```
NLabel {  
    label: "Behavior"  
}
```

```
// ...  
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Fill Width

Most controls should fill the available width:

```
NTextInput {  
  Layout.fillWidth: true  
  // ...  
}  
  
NToggle {  
  Layout.fillWidth: true  
  // ...  
}
```

Complete Example

Here's a complete settings UI example:

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Widgets

ColumnLayout {
    id: root

    property var pluginApi: null

    // Local state
    property string editMessage:
        pluginApi?.pluginSettings?.message ||
        pluginApi?.manifest?.metadata?.defaultSettings?.message ||
        "Hello World"

    property color editBgColor:
        pluginApi?.pluginSettings?.backgroundColor ||
        pluginApi?.manifest?.metadata?.defaultSettings?.backgroundColor ||
        "#A9AEFE"

    property bool editShowIcon:
        pluginApi?.pluginSettings?.showIcon ??
        pluginApi?.manifest?.metadata?.defaultSettings?.showIcon ??
        true

    property int editRefreshRate:
        pluginApi?.pluginSettings?.refreshRate ||
        pluginApi?.manifest?.metadata?.defaultSettings?.refreshRate ||
        30

    spacing: Style.marginM

    Component.onCompleted: {
        Logger.i("MyPlugin", "Settings UI loaded")
    }

    // Text input
    NTextInput {
```

Layout.fillWidth: true

label: "Display Message"

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
text: root.editMessage
onTextChanged: root.editMessage = text
}

// Color picker
ColumnLayout {
    Layout.fillWidth: true
    spacing: Style.marginS

    NLabel {
        label: "Background Color"
        description: "Widget background color"
    }

    NColorPicker {
        Layout.preferredWidth: Style.sliderWidth
        Layout.preferredHeight: Style.baseWidgetSize
        selectedColor: root.editBgColor
        onColorSelected: function(color) {
            root.editBgColor = color
        }
    }
}

NDivider {
    Layout.fillWidth: true
    Layout.topMargin: Style.marginS
    Layout.bottomMargin: Style.marginS
}

// Toggle
NToggle {
    Layout.fillWidth: true
    label: "Show Icon"
    description: "Display an icon next to the message"
    checked: root.editShowIcon
```

```
onCheckedChanged: root.editShowIcon = checked
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
ColumnLayout {
    Layout.fillWidth: true
    spacing: Style.marginS

    NLabel {
        label: "Refresh Rate"
        description: "Update interval in seconds: " + root.editRefreshRate
    }

    NSlider {
        Layout.fillWidth: true
        from: 5
        to: 120
        stepSize: 5
        value: root.editRefreshRate
        onValueChanged: root.editRefreshRate = value
    }
}

// Preview section
NDivider {
    Layout.fillWidth: true
    Layout.topMargin: Style.marginS
    Layout.bottomMargin: Style.marginS
}

NLabel {
    label: "Preview"
}

Rectangle {
    Layout.fillWidth: true
    Layout.preferredHeight: 50
    color: root.editBgColor
    radius: Style.radiusM
}
```

```
RowLayout {  
    anchors.centerIn: parent
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NIcon {  
    icon: "heart"  
    visible: root.editShowIcon  
    color: Color.mOnPrimary  
}  
  
NText {  
    text: root.editMessage  
    color: Color.mOnPrimary  
    pointSize: Style.fontSizeM  
}  
}  
  
// Save function - called by the dialog  
function saveSettings() {  
    if (!pluginApi) {  
        Logger.e("MyPlugin", "Cannot save: pluginApi is null")  
        return  
    }  
  
    pluginApi.pluginSettings.message = root.editMessage  
    pluginApi.pluginSettings.backgroundColor = root.editBgColor.toString()  
    pluginApi.pluginSettings.showIcon = root.editShowIcon  
    pluginApi.pluginSettings.refreshRate = root.editRefreshRate  
  
    pluginApi.saveSettings()  
  
    Logger.i("MyPlugin", "Settings saved successfully")  
}
```

Best Practices

1. **Use local state:** Don't modify pluginSettings directly until save
2. **Provide defaults:** Always fall back to defaultSettings from manifest

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

4. **Group related settings:** Use sections with labels and dividers
5. **Validate input:** Check values before saving if needed
6. **Show previews:** When possible, preview changes before saving
7. **Log actions:** Use Logger for debugging
8. **Handle null:** Always check pluginApi before accessing

Accessing Settings Elsewhere

Once saved, settings are available throughout your plugin:

```
// In BarWidget.qml, Panel.qml, etc.  
readonly property string message:  
    pluginApi?.pluginSettings?.message ||  
    pluginApi?.manifest?.metadata?.defaultSettings?.message ||  
    "Default"
```

See Also

- [Getting Started](#) - Create your first plugin
- [Plugin API](#) - Full API reference
- [Manifest Reference](#) - Default settings configuration
- [Translations](#) - Translate your settings UI